

Sprint 4 Deliverables

Product Interface MockUp

Interface walkthrough video that goes through our draft version of the Grocery Finder

https://drive.google.com/file/d/1FYNRIS75_j_DMSUKFIYwULGs4iDJNBGm/view?usp=sharing

User Stories

1. Fast Shopping for Students Without Cars

User Story:

As a college student who doesn't own a car, I want the app to help me efficiently shop and locate items in my cart so I can make the next bus rotation

Story Points: 20

Notes / Discussion:

- College students rely on buses or public transportation
- The app should generate the most efficient route through the store (Possibly by aisle number)
- Requires a store layout

- Could include estimated completion time (Very unlikely).

Testing:

- Input a grocery list into the app
 - The app generates an optimized path through the store.
 - The route updates once an item is obtained and checked off on the app
 - Test by comparing the time spent shopping with and without the optimized route.
-

2. Price Comparison Between Stores**User Story:**

As a college student with low funds, I want to compare my shopping cart prices between grocery stores to find the cheapest options.

Story Points: 20

Notes / Discussion:

- Requires integration with store price data or APIs
- Can show two different ways:
 - Total cart price per store
 - Price per item
- Some stores may not have updated prices

Testing:

- User inputs a grocery list
 - The website displays the estimated total price for each store
 - Individual items show cheaper alternatives at different stores
 - Testing involves comparing prices between stores by hand to see if the items are the cheapest
-

3. Coupon and Deal Suggestions

User Story:

As a student with no income, I want the app to suggest coupons, deals, and sales in the store so that I can save money.

Story Points: 8

Notes / Discussion:

- Could include in the price optimizer, but would make the price optimizer more complex
- Pull from store promotions or coupon APIs
- We could highlight sale items when the user is creating a grocery list

Testing:

- When a user adds an item to the list, available discounts appear
- Deals update automatically if promotions change
- Test by verifying deals against current store advertisements

4. Accessibility for Elderly Users

User Story:

As an elderly, retired woman, I want to efficiently and easily understand and use the app by increasing the font size and updating the font/background colors so that I can see correctly.

Story Points: 13

Notes / Discussion:

- Accessibility is important for older users
- Features may include adjustable text size or simplified UI

Testing:

- The user can increase font size within settings.
- Let someone new to the app test to see if they can make a grocery list and optimize it based on their preferences

5. Inventory Checking During Emergencies

User Story:

As a person in a natural disaster with limited time before the storm hits the area, I want to check if items are in stock and leave as quickly as possible to go home and prepare for the weather.

Story Points: 13

Notes / Discussion:

- Integrates store inventory information if available
- Have a filter that filters out unavailable items or items that are out of stock

Testing:

- User checks the availability of items before adding to the grocery list
 - If the user adds an item that is out of stock, a pop-up tells the user that the item isn't available
 - Test using simulated inventory data
-

6. Budget Optimization**User Story:**

As an unemployed person, I want to use the app's optimization function on my shopping list so that the total price is minimal.

Story Points: 20

Notes / Discussion:

- The algorithm selects the cheapest versions of items
- Could recommend certain brands of the grocery item and different stores

Testing:

- User inputs grocery list
 - The app suggests cheaper alternatives
 - Verify savings by comparing the optimized cart versus the original cart.
-

7. Prevent Overspending

User Story:

As a chronic overspender, I want to only buy things I need so that I don't buy unnecessary items.

Story Points: 5

Notes / Discussion:

- Features we could implement are spending limits and a budget tracker
- The website may choose stores with lower prices if given a spending limit if the spending is over the limit for a specific store
- The user could very well just ignore this spending limit/budget

Testing:

- User sets a spending limit
 - The app alerts them if they exceed the limit from the grocery list created, even when optimized for price
-

8. Meal Recommendation

User Story:

As a freshman in college, I want the app to recommend taco ingredients so that I can make dinner for my roommate and me.

Story Points: 40

Notes / Discussion:

- Most complex feature, since we may need some form of AI to suggest a meal or we could have a list of rotating meals that is randomly chosen
- Requires a recipe database, ingredient detection, and automatic grocery list generation

Testing:

- User selects a meal suggestion feature
 - The app generates an ingredient list for that meal
 - The ingredients integrate into the optimized store route
-

9. Fast Grocery Finding for Walmart Employees

User Story:

As a Walmart employee, I want to find groceries as quickly as possible so I can move onto the next Walmart+ order.

Story Points: 0 (already solved problem)

Notes / Discussion:

- Many stores already provide picking tools, and Walmart employees have devices that tell them where the items are
- This story could validate the usefulness of the pathfinding feature.

Supplemental Specification

Functional Requirement:

ID	Requirement Description	Priority (MoSCoW)	Source
F.1	The system must allow users to register and log in to save preferences.	Must	UC1, UC2
F.2	The system must identify the user's geographic location via ZIP or GPS.	Must	UC4
F.3	The system must generate an optimized path based on aisle location.	Must	UC3, Story 1

ID	Requirement Description	Priority (MoSCoW)	Source
F.4	The system must provide real-time pricing comparisons across stores.	Must	UC6, Story 2
F.5	The system must support item filtering based on stock availability.	Should	Story 5
F.6	The system should suggest coupons/deals based on cart items.	Should	Story 3
F.7	The system should provide automated meal ingredient suggestions.	Could	Story 8

Usability:

ID	Requirement Description	Priority	Source
U.1	Accessibility: Users must be able to adjust font sizes and toggle high-contrast/dark mode.	Must	UC7, Story 4

ID	Requirement Description	Priority	Source
U.2	Intuitive Navigation: The pathfinding UI must be simple enough for users who "get confused easily" or are "sidetracked."	Must	UC8
U.3	Efficiency: The optimized route must reduce unnecessary backtracking to save time for students and commuters.	Must	Story 4
U.4	Visual Feedback: The system must provide a real-time preview of text changes during font adjustment.	Should	UC7

Performance:

ID	Requirement Description	Priority	Source
P.1	Optimization Speed: The BFS routing logic must calculate the path within a timeframe that doesn't hinder shopping.	Must	Vision 11
P.2	UI Responsiveness: The interface must swap CSS color variables instantly when toggling dark mode.	Must	UC3

ID	Requirement Description	Priority	Source
P.3	API Efficiency: Store pricing and location updates must refresh without significant delay to the user.	Should	UC3

Reliability:

ID	Requirement Description	Priority	Source
R.1	Real-time Updates: The path must update instantly as the user checks off items to prevent aimless walking.	Must	UC3
R.2	Data Fallback: The system must use mock data or fallbacks if specific store aisle data is blocked.	Must	UC1, UC2
R.3	Theme Persistence: Dark mode and accessibility settings must be saved to the user's account profile.	Should	Sprint 4 Schedule
R.4	Credential Security: The system must notify users if login credentials are incorrect and prompt a retry.	Should	UC3, Story 1

Supportabilities :

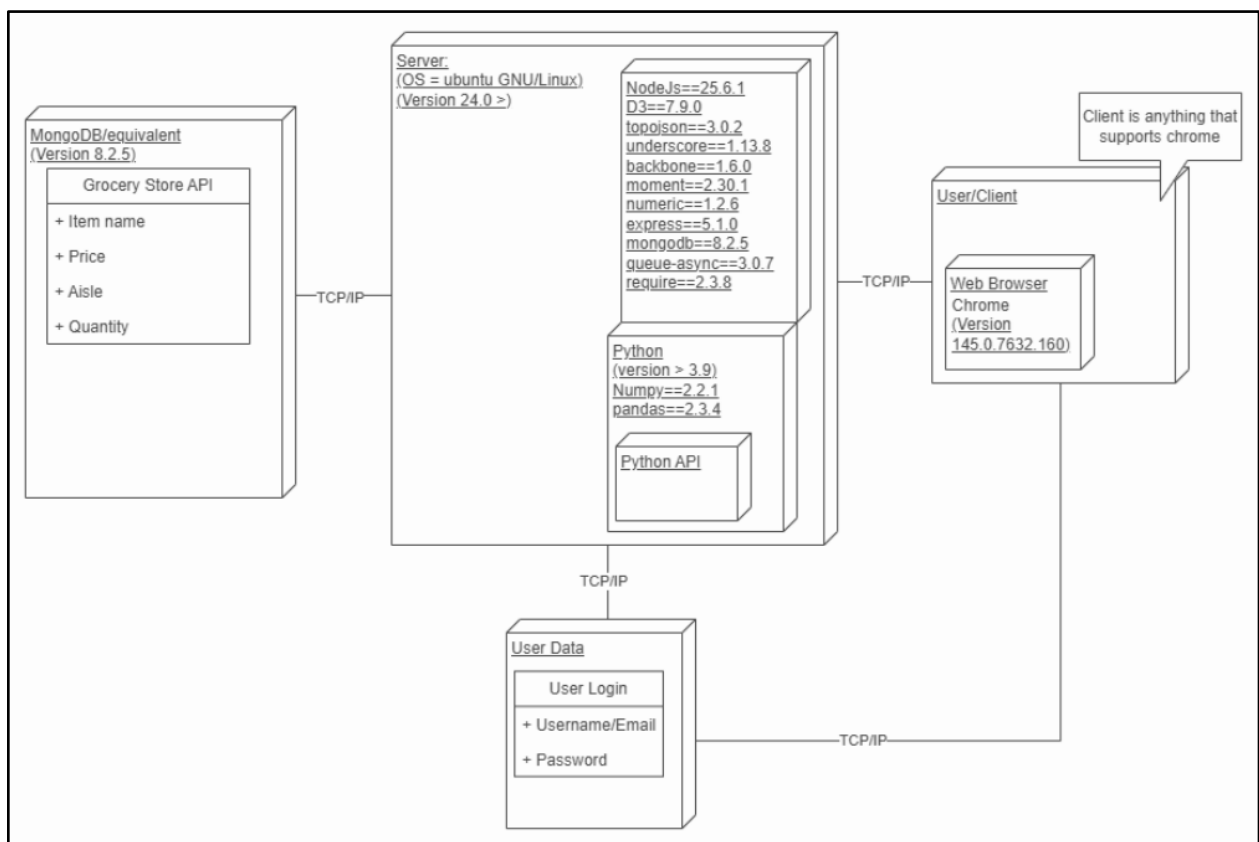
ID	Requirement Description	Priority	Source
S.1	Extensibility: The "StoreConnector" architecture must allow new store APIs to be added without modifying core logic.	Must	Sprint 6 Schedule
S.2	Maintainability: All code must be documented and hosted on GitHub with descriptive commit messages.	Must	Status Report
S.3	Event Logging: The backend must log all API search queries and pathfinding errors for troubleshooting.	Should	Sprint 2 Schedule

+ (Constraints & Interfaces) :

ID	Requirement Description	Priority	Source
I.1	Implementation: The system must be developed using a React frontend and a Python Flask backend.	Must	Major Risks

I.2	Interface: The system must integrate with external store inventory APIs (Target, Walmart, etc.).	Must	Vision 43
I.3	Design Constraint: The app must remain deployable in developer mode on local Apple hardware for testing.	Must	Sprint 3 Schedule

Deployment Diagram



Use Cases

80% of all known use cases must be written out in detail using the fully dressed format, as discussed

in class. Include a separate list of all the use cases in your project.

Use Case 1:

Use Case: userLogin	
Identifier:	UC1
Description:	The userLogin use case models how the customer could log in to their account to update their personal shopping list
Actors:	Customer
Preconditions:	The customer has an account on the website, and the website is open
Flow of events:	1. The use case starts with the customer clicking on the login 2. The website prompts the customer to input their credentials 3. The customer inputs their credentials to log in to their account 4. If the customer inputted the correct information: a. The system tells the customer that they are successfully logged in to their account. b. The system moves to the main page 5. Else a. The system tells the customer that their credentials are incorrect b. The system prompts the customer to input their login

Use Case: userLogin	
	credentials again
Post Conditions:	The user is logged in to their account with their profile picture displayed instead of login/signup, and is on the main page

Use Case 2:

Use Case: userRegistration	
Identifier:	UC2
Description:	The userRegistration use case models how a new visitor creates a customer account to save shopping preferences and track orders.
Actors:	Customer
Preconditions:	The visitor does not have an existing account, and the website is open to the registration page.
Flow of events:	1. The use case starts with the visitor clicking on the "Sign Up" or "Create Account" button. 2. The website prompts the visitor to input their details (Name, email, and password). 3. The visitor inputs their information and submits the form. 4. If the provided information meets all requirements (Unique email,

Use Case: userRegistration	
	strong password): - The system creates a new user profile in the database. - The system tells the visitor that their account was created successfully. - The system moves to the main page. 5. Else - The system tells the visitor why the registration failed. - The system prompts the visitor to correct the information and try again.
Post Conditions:	A new account is stored in the system, the user is automatically logged in, and their profile name/picture is displayed on the main page.

Use Case 3:

Use Case: optimizePath	
Identifier	UC3
Description	Takes a user's shopping cart and finds the optimal path in the store to save time shopping.
Actors	Customer

Use Case: optimizePath	
Preconditions	The customer has put all items in their grocery cart on the app and is ready to begin shopping.
Flow of events	1. The use case starts when the customer selects “Find Optimized Route”. 2. The system prompts the customer if they are entering near aisle 1 or the last aisle (whatever that number is). 3. The customer selects which aisle they are near. 4. The system starts from either the left or the right of the store, depending on the option chosen. 5. While the customer has items in the cart: a. The app prompts them to which aisle the nearest item is in. b. The customer indicates they have grabbed that item. c. That item is filtered to the bottom of the list under items grabbed.
Post Conditions	The customer’s cart is empty and they have completed their grocery shopping.

Use Case 4:

Use Case: userLocation	
Identifier	UC4
Description	The userLocation use case models how the system identifies the customer's geographic location to provide accurate local inventory or shipping estimates.
Actors	Customer
Preconditions	The customer is on the website and has reached the page requiring location data to find nearby stores.
Flow of events	1. The use case starts with the customer clicking a "Use My Location" button or entering a ZIP code. 2. The website prompts the customer for permission to access browser location services. 3. The customer either grants permission or manually enters their location. 4. If the system successfully acquires the location: a. The system tells the customer their location has been identified. b. The system updates the page to show the nearest stores or local pricing.

Use Case: userLocation	
	5. Else: a. The system tells the customer that the location could not be determined. b. The system prompts the customer to select a default region or enter a location manually.
Post Conditions	The customer's location is saved for the session, and the interface displays information relevant to that area.

Use Case 5:

Use Case: changeStore	
Identifier	UC5
Description	The changeStore use case models how a customer switches their current shopping location to a different grocery store by entering a new zip code.
Actors	Customer
Preconditions	The customer is logged in and currently has a store selected
Flow of events	1. The use case starts with the customer clicking on the current store name/location in the header.

Use Case: changeStore	
	2. The website prompts the customer to input a new zip code or change grocery stores. 3. The customer inputs a new zip code and clicks "Search." 4. If the system finds stores in the new area: a. The system displays a list of nearby stores. b. The customer selects a new store and clicks to switch to that store c. The system updates the session with the new store and refreshes the current page. 5. Else a. The system tells the customer that no stores were found for that zip code. b. The system keeps the original store selection active and prompts the customer to try again.
Post Conditions	The website interface updates to the new store's inventory, grocery locations, and pricing. Any items in the shopping list are flagged if they are unavailable.

Use Case 6:

Use Case: optimizeCost	
Identifier	UC6
Description	The optimizeCost use case models how the system analyzes a customer's shopping list and compares prices across all nearby grocery stores to identify the most cost-effective items.
Actors	Customer
Preconditions	The customer has at least one item in their shopping list and has provided a zip code
Flow of events	1. The use case starts with the customer clicking the "Optimize My Costs" button. 2. The system identifies all grocery stores within the customer's specified zip code radius. 3. The system fetches real-time pricing for every item on the customer's list from each identified store. 4. If the system finds price differences or cheaper alternatives: a. The system suggests "Split Shopping" (e.g., "Save \$12 by getting Produce at Store A and Meat at Store B"). b. The customer reviews the recommendations and selects their preferred cost-saving route.

Use Case: optimizeCost	
	5. Else (Identical pricing or item only at one store): a. The system tells the customer that they already have the best price. b. The system returns the customer to their standard shopping list view.
Post Conditions	The customer's shopping list is updated, showing which store offers the best price for each item, and the potential total savings are displayed.

Use Case 7:

Use Case: changeFontSize	
Identifier	UC7
Description	The changeFontSize use case models how a user adjusts the scale of the application's text to improve readability and accessibility.
Actors	Customer
Preconditions	The user is on the settings page of the application.
Flow of events	1. The use case starts with the user clicking on the Font Size button. 2. The system displays a slider or a set of size options (Small,

Use Case: changeFontSize	
	Medium, Large, Extra Large). 3. The user selects a new font size or moves the slider to their preferred level. 4. If the system supports the selected scale: a. The system provides a real-time preview of how text will appear. b. The user confirms the change by clicking apply, then save. c. The system updates the interface using the new font scale. 5. Else (Device constraints): a. The system tells the user that the maximum supported size has been reached. b. The system reverts to the last functional size or the default size.
Post Conditions	The updated font size is saved to the user's preferences or account profile and updated across all pages and future sessions.

Use Case 8:

Use Case: darkMode	
Identifier	UC8

Use Case: darkMode	
Description	The darkMode use case models how a user toggles the visual theme of the application between a light and dark mode.
Actors	Customer
Preconditions	The user is on the settings page
Flow of events	1. The use case starts with the user clicking the Dark mode toggle or selecting from the interface. 2. The system displays current theme options: light, dark, or system default. 3. The user selects dark mode or toggles the switch on. 4. If the system supports theme switching on the current device: a. The system instantly swaps the CSS color variables so the background is dark and the text is light. b. The system tells the user that the theme has been updated. c. The system saves this preference to the user's account. 5. Else a. The system tells the user that the theme can't be changed due to the OS settings. b. The system prompts the user to change their device settings to see a change in theme.

Use Case: darkMode	
Post Conditions	The application maintains the dark theme across all views until the user manually changes it back.

All Use Cases:

1. userLogin
2. userRegistration
3. optimizePath
4. userLocation
5. changeStore
6. optimizeCost
7. changeFontSize
8. darkMode
9. itemFilter
10. priceFilter

Work Breakdown Structure (WBS)

1. User Authentication & Profile Management

1.1. Sign-up Module

- 1.1.1. Design Sign-up Page Interface
- 1.1.2. Develop Account Creation Code
- 1.1.3. Enable and Configure MongoDB for User Data
- 1.1.4. Testing: Account Creation & Edge Case Catching

1.2. Login Module

- 1.2.1. Design Login Page Interface
- 1.2.2. Develop Login Authentication Code
- 1.2.3. Implement Credential Validation Algorithm
- 1.2.4. Testing: Login Page & Interface Integration

2. **Search & Optimization Engine**
 - 2.1. **Location Services**
 - 2.1.1. Design Zipcode/Location Input Interface
 - 2.1.2. Develop Zipcode/Location Mapping Logic
 - 2.1.3. Testing: Zipcode Store-Finding Accuracy
 - 2.1.4. Debugging: Location Filter Logic
 - 2.2. **Data Integration (APIs)**
 - 2.2.1. Implement Walmart API
 - 2.2.2. Implement Target API
 - 2.2.3. Implement Hannaford API
 - 2.2.4. API Testing: Information Pull Verification
 - 2.3. **Optimization Logic**
 - 2.3.1. Develop Cost Optimization Code
 - 2.3.2. Develop Location Optimization Code
 - 2.3.3. Implement Cost Filter Logic
 - 2.3.4. Debugging: Cost & Location Algorithms
3. **Frontend UI**
 - 3.1. **Global Interface Features**
 - 3.1.1. Develop Main Interface Framework Code
 - 3.1.2. Implement Dark Mode Code
 - 3.1.3. Implement Font Size Customization Code
 - 3.2. **UI/UX Validation**
 - 3.2.1. Functional Interface Testing
4. **Project Milestones & Deliverables**
 - 4.1. **Sprint & Interim Phases**
 - 4.1.1. Complete Sprint 4 Deliverables
 - 4.1.2. Finalize Interim Release Deliverables
 - 4.1.3. Finalize Beta Release Deliverables
 - 4.2. **Project Closure**
 - 4.2.1. Final Release Deliverables
 - 4.2.2. Develop & Deliver Final Presentation

Updated Project Schedule

Sprint #	Date	Plan	Deliverable	Interim Features to Be Developed	Est. team hour
Sprint 1	2/2-2/13	Finalize scope, split tasks, begin Vision statement, add user scenarios.	Sprint 1 docs draft, initial feature list + priority ranking.	Draft user flow for grocery list → route output.	35-50 hrs
Sprint 2	2/10 - 2/21	Pathfinding research (evaluating BFS, A*, DFS, Bellman-Ford) and framework selection.	Framework decision finalized (React for Frontend, Flask for Backend).	UI rough draft aligned with user scenarios.	35-55 hrs

Sprint #	Date	Plan	Deliverable	Interim Features to Be Developed	Est. team hour
Sprint 3	2/21 - 2/28	Finalize algorithm choice, fix Sprint 1 submission, define system architecture.	UML Diagram, 2 specific Use Cases, revised Sprint 1 docs.	Algorithm logic defined (BFS selected due to equal-cost step constraints).	45-70 hrs
Sprint 4	2/28 - 3/10	Environment setup, UI improvements based on class feedback, initial React/Flask integration.	Running app skeleton and repo scaffold.	Basic React shopping list input UI, Flask API endpoint skeleton.	55-80 hrs

Sprint #	Date	Plan	Deliverable	Interim Features to Be Developed	Est. team hour
Sprint 5	3/10 - 3/16	Implement BFS routing logic using a mock aisle map; connect frontend and backend.	End-to-end MVP (mock version) / Interim Release.	"Sort by aisle" logic working on mock data using BFS, basic data flow (input → list display → route).	60-90 hrs
Sprint 6	3/17 - 3/23	Harden integration, implement unknown-item handling, and basic error states.	Live data prototype integration.	Product/item caching layer, loading/empty/error UI states, fallback strategy for unmapped items.	55-85 hrs

Sprint #	Date	Plan	Deliverable	Interim Features to Be Developed	Est. team hour
Sprint 7	3/24 - 3/30	Multi-store foundation or improved mapping accuracy, unit testing.	Refactored connector abstraction, test suites.	Improved fallback rules (category grouping), basic end-to-end flow tests.	55-85 hrs
Sprint 8	3/31 - 4/6	UX polish, scenario-based demo script preparation, performance tuning.	Beta Release (Demo ready).	Full end-to-end user flow polished, data consistency checks.	45-70 hrs

Sprint #	Date	Plan	Deliverable	Interim Features to Be Developed	Est. team hour
Sprint 9	4/7 - 4/13	Implement quality-of-life (QoL) features based on Beta feedback.	Polished UI and QoL feature package.	Better item grouping, “Item not found” suggestion flow, edit/reorder/clear list behaviors.	45-75 hrs
Sprint 10	4/14 - 4/20	Bug bash, final testing, deployment packaging.	v1.0 Release Candidate and docs.	Final regression test checklist, deployment scripts.	45-70 hrs
Sprint 11	4/21 - 4/27	Final fixes, demo rehearsal, and submission packaging.	v1.1 and final submission package.	Final demo rehearsal polish, optional extra store support.	35-60 hrs

Project Status Report

The project has capitalized on the solid foundation constructed for Sprint 1, has made decent progress, and has created a preliminary version of the website for Sprint 4. The team has defined the project's measurables as shown in the Supplemental Specification section of this report, defining several critical features of our project that must be implemented, including but not limited to the login feature, the cost and location optimization feature, real time updates to grocery store items, etc. The team has built upon these features and created a well-drawn UML diagram that displays how our product will be constructed, as well as the types or versions of tools that the team will use to create the product. The diagram also illustrates the different components that make up Grocery Finder's functionalities. The group has also created a working draft version of the product, even though the website does currently have issues with searching for stores through a zip count or other advanced features, such as optimizing based on cost or efficiency. The draft version includes a functional login page that stores the user's information and checks if an email address is already in use to create an account. The draft version of the our product also went through several iterations of changes, mostly caused by issues encountered during the coding process, where the first version had a buggy login code that did not retain any information about the user's information and will require the user to sign up for a new account saying the email was not recognized even though the same email was used to create one minutes ago. Sebastian was the one who handled this error, as he was the one responsible for the basic UI, he handled it by fixing this bug through a simple local storage. If the team had not caught this error, it would be detrimental to the development of our product, as not being able to retain login will surely show to our customers that our product is faulty and not to be trust and it is a

relief to have caught and fixed this error as soon as possible. The team has also made decent progress when exploring a multitude of APIs that can be used in this project, which get the addresses of different stores when given a zip code, such as the Walmart API, which includes a multitude of functionality, such as store locators and a search API, which finds items sold when given the ID of a store. Overall, the team developed accordingly with our projected schedule, built on a strong foundation, and is aiming to cap off this project, a carefully crafted, bug-free product that delivers on our promises.

Contribution Summary

Sebastian Grammas:

My main contribution this sprint was implementing a few new features to the UI to match the user stories, including setting a budget, showing items in stock, suggesting coupons and meal plans, and the ability to prepare prices across stores. I also recorded all the videos for the user stories and helped write a few of them in class as well along with contributing with all the in class activities.

Peijing Xu:

My main contributions this sprint were on the UML diagram where I researched for what versions of python libraries, chrome, mongoDB, and others should be, basically what our product will support. I also wrote out the project status report summarizing the team's efforts and accomplishments throughout this sprint. Finally, I always contributed for in-class activities as well as for the weekly check-in meeting with Namir.

Alan Lee:

My main contributions this sprint were expanding and completing the user stories, use cases, and work breakdown structure for our product from the work done in class. I also started

attempting to incorporate APIs into our product. Outside of technical work, I helped with leading the meetings outside of the one we have with Namir, by checking in with group members on what needs to be completed or what the current progress on a task is, and contributing with in-class activities.

Qizheng Sui:

My contributions focused on project scheduling, the Supplemental Specification, and team coordination. I helped update the project schedule so it better matched our current goals and release plan, and I also contributed to the Supplemental Specification by helping define important system details and requirements. In addition, I helped keep the team organized through meetings and general project support.